ITM UNIVERSITY
GWALIOR • MP • INDIA
"CELEBRATING DREAMS"

Database Management Systems – (CSL0311)
B.Tech. CSE(III Sem)
Design and Implementation of an AI-Assisted, Industry-Ready Database System

Course	Assignment Type	Team Size	Duration
DBMS	Problem-Based and Project-Based Learning (PbBL)	3-4 students	8-10 weeks

Course Outcomes:

CO1 Understand and implement Modern DB concepts AI-assisted DB modeling &n design (BL2)
CO2 Understanding and implementing Databases using Languages SQL + Relational Algebra (BL3)
CO3 Designing and fine tuning efficient Databases by applying Normalization Principles (BL5)

CO4 Understanding and applying concept of Transactions and Concurrency control and fine tune the databases to suit the distributed environment (BL5)

CO5 Design and implement databases for some real life Intelligent DB applications (BL6)

1. Assignment Overview

Each team will identify a real-world organizational problem and develop a complete database solution covering problem analysis, requirement engineering, conceptual design, logical design, normalization, physical design, implementation, testing, security, optimization, deployment, monitoring, migration, backup and future evolution.

The final solution must be fully designed, implemented, tested, secured, optimized, deployed and documented. It must also use contemporary industry practices such as database-as-code, schema version control, migration scripts, automated testing, Database DevOps, cloud or container deployment, observability and responsible use of AI-assisted tools.

2. Problem Statement

Modern databases must do more than store records. They must ensure accuracy, consistency, reliable transactions, secure access, scalability, acceptable response time, maintainability, controlled schema evolution, recoverability and support for analytics or AI-enabled applications.

The proposed system must support operational transactions, management reporting, role-based security, performance optimization, backup and recovery, database migration, and at least one modern or AI-enabled database feature.

3. Suggested Project Domains

- Smart University Resource Management
- Hospital and Patient-Care Management
- Agriculture Supply-Chain Management
- E-Commerce and Inventory Management
- Cybersecurity Incident and Threat-Intelligence Repository
- Smart City Complaint Management
- Research Grant and Publication Management
- Electric Vehicle Charging Management
- Placement and Skill-Intelligence Platform
- Disaster-Relief Resource Coordination

4. Driving Question

How can a secure, scalable, maintainable and AI-ready database be designed and implemented for a real-world organization using contemporary industry practices?

5. Project Objectives

- Analyse a real-world problem and stakeholder requirements.
- Prepare conceptual, logical and physical database designs.
- Apply normalization and integrity constraints.
- Implement queries, views, procedures, functions and triggers.
- Design reliable transactions and concurrency controls.
- Apply security, privacy, auditing and data governance.
- Optimize performance using indexes and execution plans.
- Manage schema versions, migrations and rollback scripts.
- Automate database testing and deployment.
- Use AI tools responsibly during design, coding, testing or migration.
- Deploy using cloud or container technology.
- Prepare backup, recovery, observability and maintenance plans.
- Document and defend all important design decisions.

6. Expected PBL Outcomes

S.No.	Learning Outcome
1	Analyse stakeholder requirements and translate them into a database specification.
2	Develop conceptual, logical and physical models using appropriate normalization principles.
3	Implement database objects, constraints, transactions, queries and stored programs.
4	Apply database security, concurrency control, backup and recovery mechanisms.

5	Evaluate and optimize database performance using indexes and execution plans.
6	Apply modern Database DevOps, cloud and AI-assisted database engineering practices.

7. Mandatory Industry Practices (Preface)

7.1 Database as Code

The complete database must be maintained in a Git repository. The repository should include schema scripts, constraints, views, indexes, procedures, functions, triggers, roles, privileges, seed data, tests, migration scripts, rollback scripts and deployment instructions.

7.2 Schema Migration and Evolution

At least five incremental schema versions must be maintained, for example: V1 core tables, V2 constraints, V3 transaction module, V4 performance improvements, V5 security and audit, and V6 analytical or AI extension.

Flyway, Liquibase, Alembic, Prisma Migrate or manually versioned SQL scripts may be used. Every migration must record its version, purpose, forward script, rollback strategy, test evidence, author and date.

7.3 Automated Testing

Tests must cover primary and foreign keys, CHECK constraints, duplicate or missing data, invalid values, procedures, functions, transaction rollback, unauthorized access, business rules, migration success and expected query results.

7.4 Security by Design

The database must demonstrate least-privilege access, separate administrator and application roles, role-based access control, parameterized queries, SQL-injection prevention, restricted views, sensitive-data identification, auditing, anonymization or masking where required, and secure backup practices.

7.5 Performance-Aware Design

Students must estimate users, transaction volume, data growth, peak workload, frequently executed queries, reporting workload and retention period. These estimates should guide indexing, partitioning, storage selection, caching, archival and any controlled denormalization.

7.6 Hybrid Storage Decision

Teams must evaluate whether the application needs relational tables, JSON/document storage, graph data, geospatial data, time-series data, key-value storage, vector embeddings or object storage. Multiple databases should be adopted only when technically justified.

7.7 Observability

The monitoring plan should cover slow queries, failed transactions, connection count, storage utilization, deadlocks, lock waits, index usage, backup status and data-quality failures.

8. Database Design and Development Lifecycle

Phase 1: Problem Identification

- Select a real-world problem and study the existing system.
- Define scope, assumptions, constraints, feasibility and expected benefits.

Key deliverable: Project charter, problem statement, organization profile, existing/proposed system and scope document.

Phase 2: Requirement Analysis

- Identify at least four stakeholder categories.
- Prepare at least 12 functional requirements, 8 non-functional requirements, 15 business rules and 8 user stories.
- Develop acceptance criteria, use cases, stakeholder matrix and CRUD matrix.

Key deliverable: Requirement specification, stakeholder analysis, business rules, use cases and acceptance criteria.

Phase 3: Data Discovery and Profiling

- Identify data sources and prepare sample or synthetic data.
- Detect missing, duplicate and inconsistent records.
- Classify sensitive data and define cleaning and quality rules.
- Use at least 1,000 rows in the main transaction table, 100 rows in major master tables and approximately 10,000 total rows, or justify an alternative.

Key deliverable: Data-source inventory, profiling report, data dictionary, cleaning rules, sensitive-data classification and cleaned sample dataset.

Phase 4: Conceptual Design

- Create an ER model with at least 10 major entities.
- Include at least two many-to-many relationships, one weak/dependent entity and one recursive, ternary, inheritance or temporal relationship.
- Specify attributes, identifiers, cardinality and participation constraints.

Key deliverable: Initial and revised ER diagrams, entity/relationship descriptions, assumptions and rule-to-model traceability.

Phase 5: Logical Design and Normalization

- Convert the ER model to relations and identify primary, alternate and foreign keys.
- Normalize at least two relations from unnormalized form through 1NF, 2NF, 3NF and BCNF where applicable.
- Document functional dependencies, candidate keys, anomalies and decomposition decisions.

Key deliverable: Relational schema, logical ER model, normalization worksheets, dependency analysis and logical data dictionary.

Phase 6: DBMS Selection

- Compare at least three platforms using transaction support, integrity, security, indexing, JSON/vector support, scalability, backup, cloud availability, cost and administration complexity.

Key deliverable: Technology comparison, weighted decision matrix, selected DBMS justification and deployment architecture.

Phase 7: Physical Design

- Define DBMS-specific data types, naming standards, indexes, identity/sequence fields, storage estimates, partitioning or archival and retention strategy.

Key deliverable: Physical schema, DBMS-specific ER diagram, DDL scripts, index design and storage estimation.

Phase 8: Implementation

- Implement 10-15 related tables, 15 key constraints, 10 CHECK/domain constraints, 5 additional indexes, 5 views, audit tables, seed data and a schema-version table.
- Prepare at least 20 meaningful queries including joins, subqueries, correlated subqueries, aggregation, GROUP BY/HAVING, set operations, CTEs, window functions, ranking, date functions, conditional expressions, views, JSON operations where relevant and transaction-control commands.
- At least five queries must answer management or strategic questions.

Key deliverable: Complete executable database and query portfolio.

Phase 9: Transactions and Concurrency

- Define at least three important business transactions with boundaries, validation, affected tables, isolation level, possible failures, rollback behaviour and concurrency risks.
- Demonstrate COMMIT, ROLLBACK, SAVEPOINT, conflicting/lost update, lock wait or deadlock and an appropriate isolation-level solution.

Key deliverable: Transaction design and concurrency demonstration report.

Phase 10: Performance Optimization

- Select at least five important queries and record original query, plan, execution time, bottleneck, optimization, revised plan, revised time and percentage improvement.
- Use indexes, query rewriting, join improvement, partitioning, materialized views or justified denormalization as appropriate.

Key deliverable: Before-and-after performance report with evidence.

9. AI-Assisted Database Engineering

Each team must use AI tools in at least three different project stages. Suitable tools may include ChatGPT, GitHub Copilot, Gemini, Claude, Amazon Q Developer, Microsoft Copilot or AI-enabled database development environments.

Project Stage	Permitted AI Applications
Requirements	Generate stakeholder questions, classify requirements, identify ambiguities and

	prepare user stories.
Conceptual Design	Suggest entities/relationships, review ER diagrams, detect missing cardinalities or redundancy.
Logical Design	Identify candidate keys, dependencies, anomalies and review normalization.
SQL Development	Generate initial SQL, procedures, constraints, test data and test cases.
Performance	Explain execution plans, suggest indexes and rewrite inefficient queries.
Migration	Convert SQL dialects, map data types, identify incompatibilities and suggest rollback tests.
Documentation	Draft data dictionary descriptions, code explanations and deployment instructions.
Intelligent Feature	Natural-language-to-SQL, semantic search, recommendation, anomaly detection or duplicate detection.

10. AI Usage and Validation

Teams must maintain an AI usage log recording date, project stage, tool, prompt summary, output used, validation method, modifications and responsible member.

Date	Stage	AI Tool	Prompt Summary	Validation	Changes	Member

Every AI-generated artifact must be validated through manual review, syntax checking, execution in a test database, business-rule comparison, test cases, normalization analysis, security review, execution-plan analysis, official documentation or peer review.

Students must not expose credentials or confidential data, submit untested AI-generated SQL, fabricate results, hide AI usage or accept AI-generated models without critical review.

11. AI Output Audit

Each team must formally audit one AI-generated artefact such as an ER model, normalization solution, SQL query, stored procedure, index suggestion, migration script or security recommendation.

1. Original prompt
2. AI-generated output
3. Errors or omissions identified
4. Corrected version
5. Test cases and validation evidence
6. Lessons learned about AI reliability

12. Database DevOps (Optional)

The repository should follow a clear, maintainable structure similar to the following:

```
project/
  README.md
  docs/
  database/
```

schema/
migrations/
rollback/
procedures/
functions/
triggers/
views/
indexes/
security/
seed/
tests/
ai-log/
application/
deployment/

Git requirements include meaningful commit messages, feature branches, code review, issue tracking, release tags and strict exclusion of passwords or secrets from the repository.

13. Reports and Analytics

The database must produce at least 5 operational reports, 5 management reports, 3 trend-analysis reports, 2 exception reports and 1 executive dashboard.

The team must explain whether reports use live transactional tables, views, materialized views, summary tables, analytical replicas or a data-warehouse extension.

14. Ethical and Responsible Data Use

The report must discuss minimum data collection, privacy, consent, retention, fairness, algorithmic bias, human oversight, explainability, secure deletion and responsible use of synthetic data.

For AI-generated decisions, the system should record model or rule version, timestamp, input reference, output, confidence score where available, human correction and final decision.

15. Project Schedule

Week	Milestone	Major Output
1	Problem selection	Project charter and scope
2	Requirement analysis	Requirements and business rules
3	Conceptual design	ER model
4	Logical design	Schema and normalization
5	Physical design	DDL, indexes and storage plan
6	Core implementation	Tables, constraints, views and queries
7	Testing and optimization	Test and performance reports
8	Final demonstration	Report, product and viva

16. Mandatory Deliverables

- Project proposal and requirement specification

- Stakeholder, business-rule, use-case and acceptance-criteria documents
- Conceptual and logical ER diagrams
- Normalization worksheets and relational schema
- Physical schema, data dictionary and complete database scripts
- Query portfolio and transaction/concurrency demonstrations
- Security, audit and data-governance evidence
- Test portfolio and performance-optimization report
- AI usage log and AI output audit
- Git repository
- Final project report, presentation, demonstration video and individual reflection/contribution record

17. Assessment Scheme

Assessment Component	Marks
Problem identification and requirement analysis	2
Conceptual design	2
Logical design and normalization	2
Physical design and DBMS selection	2
Database implementation	2
Transactions and concurrency	2
Testing, data quality and optimization	2
AI integration and AI audit	2
Database DevOps, migration and deployment(Optional)	3(bonus)*
Documentation and presentation	2
Individual viva and contribution	2
Total	20

18. Minimum Acceptance Criteria

1. Complex real-world problem and stakeholder-driven requirements
2. Valid ER model and normalization to at least 3NF
3. Appropriate integrity constraints and meaningful test data
4. Complex SQL, procedures, functions and triggers
5. ACID transaction handling and concurrency analysis
6. Role-based security and audit logging
7. Measured performance optimization
8. Backup and recovery demonstration
9. Version-controlled schema with migration and rollback scripts
10. Responsible, disclosed and verified AI usage
11. Complete documentation, successful demonstration and individual viva

19. Final Demonstration Sequence

1. Present the problem, stakeholders and architecture.
2. Explain the ER model and important design decisions.
3. Show the Git repository and create/deploy the database using scripts.
4. Apply a schema migration.
5. Execute a major business transaction.
6. Demonstrate a constraint violation, rollback and concurrency case.
7. Demonstrate role-based access and audit records.
8. Execute management reports.
9. Compare query performance before and after optimization.
10. Demonstrate the selected modern or AI-enabled feature.
11. Present the AI usage log and AI audit.
12. Demonstrate backup and recovery.
13. Discuss limitations and future scope.

20. Conclusion

This PBL assignment requires students to treat the database as a professionally engineered product rather than a collection of tables and SQL queries.

Problem -> Requirements -> Business Rules -> Data Model -> Schema -> Implementation -> Transactions -> Security -> Testing -> Optimization -> Deployment -> Monitoring -> Evolution

Every important design decision must be linked to a requirement, technically justified, tested, documented and defended during the viva. AI usage will be evaluated on responsible application, critical validation and improvement of AI-generated outputs rather than the quantity of generated content.